

cgroups-rs 架构重构与 v1.0.0 正式版发布项目申请书

一、项目概述

cgroups-rs 是 Rust 生态中用于管理 Linux Control Groups 的基础库，支持 cgroup v1 和 cgroup v2，并被 Kata Containers 等容器基础设施项目使用。

当前 cgroups-rs 已经具备 cgroupfs 与 systemd 两种管理方式，也形成了初步的 Manager 接口，但核心接口仍直接依赖 OCI Spec 数据结构，底层文件系统操作、systemd D-Bus 逻辑、资源配置转换和特定业务场景逻辑之间的边界不够清晰。随着代码和使用场景的增长，这些问题提高了维护成本，也阻碍了 API 稳定化和 v1.0.0 版本发布。

本项目计划在保持现有主要功能和下游可迁移性的基础上，将 cgroups-rs 重构为纯 Linux cgroup 管理库，最终形态为：同时完善测试、文档、API 兼容检查和发布流程，为 cgroups-rs 达到 v1.0.0 release-ready 状态提供工程基础。

二、项目背景与意义

2.1 项目背景

cgroups-rs 提供了创建、配置、查询和删除 Linux cgroup 的 Rust API，主要涉及：

- cgroup v1 与 cgroup v2 文件系统接口；
- systemd D-Bus 接口；
- CPU、内存、I/O、cpuset、PIDs 等资源控制器；
- OCI Runtime Spec 中 LinuxResources 到 cgroup 配置的转换；
- 进程和线程迁移、资源统计及冻结等操作。

该库属于容器运行时中的基础组件。其接口稳定性、错误处理质量和跨 cgroup 版本兼容性会直接影响上层运行时的可靠性。

2.2 当前基线与前期工作

cgroups-rs 当前主线代码已经具备：

- 默认构建下可用的低层 fs API，以及 fs::Resources、CpuResources、MemoryResources 等不依赖 OCI Spec 的公开资源类型；
- 开启 oci feature 后可用的 Manager、FsManager 和 SystemdManager；
- GitHub Actions 中的 Format、Clippy 和测试任务；
- 在固定 Ubuntu QEMU 虚拟机中运行的 cgroup v1/v2 测试矩阵，两种 hierarchy 均覆盖 cgroupfs 和 systemd 路径。

2.3 当前主要问题

2.3.1 SystemdManager 直接依赖具体 FsManager，控制面与观测面边界不清晰

当前 `SystemdManager` 的定义为：

```
pub struct SystemdManager<'a> {  
    /// The name of slice  
    slice: String,  
    /// The name of unit  
    unit: String,  
    /// Systemd client  
    systemd_client: SystemdClient<'a>,  
    /// Cgroupfs manager  
    fs_manager: FsManager,  
}
```

其中 `SystemdClient` 保存 unit 名称和待提交的 properties，本身并不是已经建立的 D-Bus 连接；实际 system bus 连接在具体操作中创建。`SystemdManager::new()` 会初始化文件系统侧的 hierarchy/path 状态和 systemd 客户端状态，并依赖 `manager::fs` 中的路径辅助函数：

```
pub fn new(path: &str) → Result<Self> {  
    let (slice, unit) = Self::parse_slice_and_unit(path)?;  
    let props = PropertiesBuilder::default_cgroup(&slice, &unit).build();  
    let slice_base = expand_slice(&slice)?;  
    let fs_base = join_path(&slice_base, &unit);  
    let fs_manager = FsManager::new(&fs_base)?;  
    let cgroup = SystemdClient::new(&unit, props)?;  
    Ok(Self {  
        slice,  
        unit,  
        fs_manager,  
        systemd_client: cgroup,  
    })  
}
```

当前 `Manager` 的 13 个方法中，有 7 个由 `SystemdManager` 直接委托给 `FsManager`：

`pids()`、`cgroup_path()`、`enable_cpus_topdown()`、`stats()`、`paths()`、`mounts()` 和 `v2()`。控制面和观测面通过具体 `FsManager` 直接耦合，无法分别测试、替换或表达失败边界。

2.3.2 已有统一基础抽象，但版本差异仍跨层泄露

当前已通过 `Controller`、`Hierarchy`、`Cgroup` 和 `fs::Resources` 对 v1/v2 的一部分差异进行了抽象，但文件名、值转换、能力差异和策略判断分散在低层控制器、高层 `Manager`、systemd property 构造以及下游调用方中。

例如高层 `FsManager` 负责 OCI CPU shares 到 cgroup v2 weight 的转换：

```
fn set_cpu(&self, linux_cpu: &LinuxCpu) → Result<()> {
    if let Some(shares) = linux_cpu.shares() {
        let shares = if self.v2() {
            conv::cpu_shares_to_cgroup_v2(shares)
        } else {
            shares
        };
        controller.set_shares(shares)?;
    }
    // ...
}
```

低层 Controller 又根据版本选择写入文件：

```
fn add_task(&self, pid: &CgroupPid) → Result<()> {
    let mut file_name = "tasks";
    if self.is_v2() {
        file_name = "cgroup.threads";
    }
    // ...
}
```

2.3.3 高层 Manager 与 OCI Spec 耦合，且后端能力不对称

现有高层 Manager 直接接收 OCI Spec 的 `LinuxResources`，非 OCI 使用者若需要统一的 cgroupfs/systemd Manager，只能引入 OCI 类型。

2.3.3.1 Manager 模块受 `oci` feature 控制

```
#[cfg(feature = "oci")]
pub mod manager;
#[cfg(feature = "oci")]
pub use manager::{FsManager, Manager, SystemdManager};
```

默认构建仍然可以使用低层 `fs` API 和现有 `fs::Resources`，但当前没有不依赖 OCI 的高层统一 Manager。

2.3.3.2 Manager 方法签名直接依赖 OCI 类型

```
pub trait Manager: Send + Sync {
    fn set(&mut self, resources: &LinuxResources) → Result<()>;
}
```

2.3.3.3 两个后端对资源字段的支持范围不同

不同Manager支持的能力不同，目前 `FsManager::set()` 处理 CPU、内存、PIDs、block I/O、hugetlb、network 和 devices，而 `SystemdManager::set()` 仅处理 CPU/cpuset、内存和 PIDs。对于其他字段，systemd 路径会直接忽略。

内存路径也存在策略差异：FsManager v2 会读取当前 usage 并提前拒绝部分 limit，而 SystemdManager 主要依赖 systemd/内核返回结果。该差异应在设计阶段确认哪些属于 OCI 输入校验、哪些属于后端约束，不能预先假设某一个后端的策略就是唯一正确语义。

2.3.3.4 资源配置测试与 OCI 类型绑定

manager 模块整体只在开启 `oci feature` 时编译，资源配置测试普遍使用 `LinuxCpuBuilder`、`LinuxMemoryBuilder` 和 `LinuxResourcesBuilder`。但 unit 名称、路径和生命周期等 manager 测试并不都通过 OCI Builder 构造。src/fs 低层实现本身不引用 `oci_spec`，并使用自有 `fs::Resources`。测试应该充分覆盖不同feature下的功能。

2.3.4 后端能力和路径语义在公共接口中表达不清晰

当前公共接口同时暴露后端类型、cgroup 版本和 v1 hierarchy 内部数据：

```
pub trait Manager: Send + Sync {
    fn systemd(&self) → bool;
    fn v2(&self) → bool;
    fn paths(&self) → &HashMap<String, String>;
    fn mounts(&self) → &HashMap<String, String>;
}
```

该接口把后端身份、cgroup 版本和 v1 hierarchy 内部数据同时暴露在公共 trait 中。调用方必须根据 `systemd()` / `v2()` 的实现身份自行分支才能确定行为（例如线程迁移在 `systemd` 与 `v1/v2` 组合下的语义不同），但真正需要的是操作能力而非身份信息；`paths()` / `mounts()` 把 hierarchy 探测结果（`HashMap<String, String>`）直接暴露在公共接口中；`cgroup_path(Option<&str>)` 返回语义不一致：v2 返回完整路径，v1 返回 subsystem 相对路径，调用方还需自行拼接 mountpoint。新接口应以运行时 capability 和语义稳定的路径类型替代这些实现细节。

2.3.5 文件系统访问、数据解析与业务语义混合

部分控制器实现同时承担路径拼接、文件读写、格式解析、版本判断和错误转换等职责，且解析函数直接绑定真实文件路径常量，无法脱离内核环境测试：

```
const CGROUP_PATH: &str = "/proc/self/cgroup"; // 硬编码路径
const MOUNTINFO_PATH: &str = "/proc/self/mountinfo";

fn parse_cgroup_subsystems() → Result<HashMap<String, String>> {
    let data = fs::read_to_string(CGROUP_PATH) // 无法注入任意输入
        .map_err(|err| FsError::with_cause(FsErrorKind::FsError, err))
        .map_err(Error::Cgroupfs)?;
```

```
// ...
}
```

同一份 `/proc/self/cgroup` 格式在 `manager/fs.rs` 与 `fs/cgroup.rs` 中存在两份行为不同的解析器：前者使用 `split(':')` 并要求恰好三段，后者使用两次 `split_once(':')` 以保留路径中的冒号。`parse_cgroup_mountinfo()` 还会在校验分段长度前直接索引 `parts[1]` 和 `part1[4]`，畸形输入可能触发 `panic`。`fs/mod.rs` 中多处读取错误仍使用 `"FIXME: read_string_from"` 作为上下文。将“读取来源”与“纯字符串解析”拆开，可以在普通单元测试中覆盖这些情况。

2.3.6 特定场景策略侵入通用库

针对 Docker-in-Docker 等特定部署环境的逻辑直接成为核心库的隐式行为，例如 `FsManager::subcgroup()`：

```
pub fn subcgroup(&self) → &str {
    // 通过"存在 init 子目录"猜测当前是否在 Docker-in-Docker 环境中
    let is_dind = self.v2() && init_exists;
    if is_dind {
        "/init/" // ← 硬编码 DinD 返回路径
    } else {
        "/"
    }
}
```

该启发式探测结果会被 `SystemdManager::add_proc()` 用作 D-Bus 进程挂入的子路径。更好的设计应是把隐式猜测改成显式 `subcgroup` 配置或调用参数，而不是进行启发式探测。

2.3.7 错误处理与 API 一致性不足

部分历史接口存在忽略错误、上下文不足、静默默认值和 `panic` 风险，例如：

```
// PID 解析失败被替换为 0
let n = line.trim().parse().unwrap_or(0u64);
// 控制器启用失败被忽略
let _rest = fs::write(f.as_path(), body.as_bytes());
// D-Bus 错误被当作 unit 不存在
let sys_proxy = match systemd_manager_proxy() {
    Ok(proxy) ⇒ proxy,
    _ ⇒ return false,
};
// hierarchy 缺失或路径无 parent 时 panic
.mount_point.parent().unwrap()
```

PID 0 会进入 `procs() / tasks()` 返回列表；`Manager::stats()` 不返回 `Result`，内部大量 `.ok() / unwrap_or_default()` 把控制器缺失、权限和解析错误转换为空统计或 0。crate 内还存在 `fs::error`、`manager::error`、`systemd::error`、`systemd::dbus::error` 四

套错误类型；部分包装保留了 source chain，但错误分类、操作上下文和"错误还是默认值"的处理策略在模块间仍有差异。

2.3.8 变更路径未受校验，且 Cgroup::clone() 会改变对象语义

ErrorKind::InvalidPath 声称阻止了 .. 逃逸，但实际上没有有效的路径校验：Cgroup::load() 接受任意路径字符串（src/fs/cgroup.rs），控制器路径通过 PathBuf::push 拼接，绝对路径会直接替换基路径，.. 组件可能逃出预期的 cgroup 根目录。所有涉及文件系统变更的入口都应只接受受校验的 CgroupPath 类型。

Cgroup::clone() 会重新执行 hierarchy 自动发现并丢弃 specified_controllers（src/fs/cgroup.rs 中 Clone 实现重新调用 hierarchies::auto()，specified_controllers 置为 None），导致克隆对象的行为与原件不同。重构后应保留 Arc<CgroupTopology> 与完整的选择状态，或移除 Clone。

2.3.9 测试、文档和稳定性保障仍需完善

现有部分测试仍通过 skip 宏选择 hierarchy（QEMU workflow 已分别启动 v1/v2 guest 并检查没有意外跳过）；剩余问题主要是纯解析逻辑绑定真实 /proc 路径，以及大量控制器错误路径只能在高成本集成环境中触发。

三、项目目标

1. 引入受校验的 CgroupPath（拒绝绝对路径、..、根路径与平台前缀组件），使所有文件系统变更入口只接受该类型；引入受检查的 ProcessId / ThreadId，拒绝零值与越界转换。
2. 审计现有 fs::Resources、资源子类型、CgroupStats 和 FreezerState，通过提升、复用或兼容转换形成与 OCI Spec 和具体后端无关的核心模型（ResourcePatch、Capabilities、可失败的统计），避免并存三套资源表示。
3. 将 mountinfo/cgroup 成员关系解析统一发现流程，形成不可变的 CgroupTopology 快照并通过 Arc 共享，取代各模块重复且行为不一致的硬编码探测。
4. 建立两个公共后端 CgroupFsManager 与 SystemdManager 及共用的 ManagedCgroup 契约；cgroup v1/v2 只作为各后端私有的 dialect 实现，不导出 FsV1Manager / FsV2Manager / SystemdV1Manager / SystemdV2Manager。
5. 分离 systemd unit 控制面与 cgroupfs 路径、PID、统计等观测/操作能力，通过显式接口组合代替对具体 FsManager 的直接依赖。
6. 将 OCI Spec 转换逻辑移动到独立适配层（仅翻译、不做 I/O、不选择路径、不检测运行时），核心接口不再直接依赖 LinuxResources。
7. 将 DinD subgroup 等隐式场景探测改为显式 AttachDestination::Subgroup / AttachDestination::SameCgroupAs(pid) 配置，并验证下游迁移。
8. 明确 unsupported 语义与结构化错误（VersionMismatch、Unsupported、PartialApply 等），禁止把失败转换为 false、空列表或零值。

9. 在核心重构完成的前提下，修复经维护者确认且没有活跃 PR 覆盖的代表性 Issue；该项为扩展目标，不阻塞主线架构验收。
10. 核心模块单元测试覆盖率达到 80% 以上，且常规单元/契约测试不需要 root 权限。
11. 完善 API 文档、使用示例、迁移指南和发布检查流程。
12. 协助维护者完成 v1.0.0 API 审计和发布准备；

四、总体架构设计

公共 API 按"管理机制"拆分 (cgroupfs / systemd 两个后端)，私有实现按"cgroup 版本"拆分 (每后端内部各含 v1/v2 两个 dialect)，形成**两个公共 Manager、四套受测实现**：

```
公共 API
|-- CgroupFsManager
|   |-- 私有 dialect: cgroupfs v1 / cgroupfs v2
|-- SystemdManager
    |-- 私有 dialect: systemd-on-v1 / systemd-on-v2
```

4.1 Core 核心模型层

核心模型与具体后端、OCI Spec 无关，只描述 cgroup 操作"是什么"，不关心"怎么做"。对现有 `fs::Resources` 等类型逐项审计后提升、复用或兼容转换。核心模型预计包含：

- 受校验的 `CgroupPath`：相对 cgroup 根目录的逻辑路径，拒绝绝对路径、`..`、根路径和平台前缀，保留为 `Path / OsStr`，可显式转换为 systemd unit 或 subgroup；
- 受检查的 `ProcessId(u32) / ThreadId(u32)`：替代当前 `CgroupPid { pid: u64 }` 直接截断写入 D-Bus 的做法；
- cgroup 原生 `ResourcePatch`：以 `Update<T> { Unchanged, Set(T), Max }` 表达明确更新语义，替代 `Option / 0 / -1` 的模糊组合；v1 shares → v2 weight 等版本差异转换集中实现并单元测试；
- `Capabilities`：不可变运行时能力快照（版本、已检测控制器、thread attach、freeze、kill、递归统计等），供操作前预检；
- 可失败的统计：`fn stats(&self, request: &StatsRequest) → Result<CgroupStats>`，部分结果用 `ControllerStats<T> { value, error }`，使控制器缺失、权限错误、解析失败、真实零值等可区分；
- `AttachDestination`：`ManagedCgroup / Subgroup(CgroupPath) / SameCgroupAs(ProcessId)`，API 中不出现 Docker、DinD、Kata、runc 或 OCI 术语；
- 结构化错误：`InvalidPath`、`InvalidResource`、`NotFound`、`AlreadyExists`、`VersionMismatch`、`Unsupported`、`PartialApply(ApplyReport)`、`Backend { backend, operation, target, source }`。

4.2 发现层 (Discovery)

建立统一发现流程，读取 `/proc/self/mountinfo`、`/proc/self/cgroup`（或其他显式选择进程）、`cgroup v2` 控制器文件及可配置的 `procfs/cgroupfs` 根路径，返回不可变的 `CgroupTopology`（`version`、`mounts`、`membership`、`controllers`），配合 `HostPaths`（`procfs`、`cgroupfs override`、`hugepages`）。

要求：覆盖 `v1/v2/hybrid` 布局；支持 `mount namespace` 与非标准挂载点；正确解码 `mountinfo` 转义字段；允许路径含冒号；畸形记录不 `panic` 并返回发现错误；正常操作不重复发现。拓扑通过 `Arc` 在 `Manager` 与 `cgroup` 句柄间共享。

4.3 统一接口层

只把两个后端都能保持语义的操作放进公共契约：

```
pub trait ManagedCgroup: Send + Sync {
    fn apply(&mut self, resources: &ResourcePatch) → Result<ApplyReport>;
    fn attach_process(&mut self, pid: ProcessId, destination:
AttachDestination) → Result<()>;
    fn processes(&self) → Result<Vec<ProcessId>>;
    fn stats(&self, request: &StatsRequest) → Result<CgroupStats>;
    fn freeze(&self) → Result<()>;
    fn thaw(&self) → Result<()>;
    fn capabilities(&self) → &Capabilities;
}
```

创建/删除、`unit stop`、原始属性访问、线程挂入和 `kill` 因所有权或副作用差异不放入契约，由具体 `Manager` 提供并返回 `Unsupported`。调用方根据 `Capabilities` 预检，不再根据 `systemd()` 等实现身份猜测行为。两个 `Manager` 同时提供同名 `inherent` 方法。

4.4 cgroupfs 驱动层

`CgroupFsManager` 通过 `CgroupFsConfig` { `path`, `version`, `controllers`, `host_paths` } 构造：`new` 只做发现与校验，`open` 要求目标已存在，`create` 不作为 `apply` / 挂入的隐式副作用（`CreateOptions` 显式定义已存在目标与缺失父目录的处理）。公共操作还包括 `attach_thread`、`threads`、`kill/remove`、`location`、`read_attribute/write_attribute` 转义口（`CgroupAttribute` 只接受一个已校验文件名组件，访问限于被管理 `cgroup` 之下）。

- `processes()` 读 `cgroup.procs`，`threads()` 读 `tasks`（`v1`）/ `cgroup.threads`（`v2`），`membership` 操作不再经由任意资源控制器；
- 线程挂入绝不静默退化为整进程挂入，无法保持 `thread-only` 语义时返回 `Unsupported`；
- `CgroupFsLocation` 以 `V1` { `paths`: `ControllerPathMap` } / `V2` { `path`: `ResolvedCgroupPath` } 表达 `v1` 多路径与 `v2` 统一路径；
- 私有 `FsDialect`（`v1/v2`）负责创建、`plan_resources`、挂入、`membership`、统计、`freeze/kill/remove` 和 `capabilities`；`plan_resources` 输出已校验的 `FsWritePlan`，先规划后执行，使校验与 `unsupported` 字段在第一次变更前暴露；

- 可注入的 `FsIo` 接口执行有界读/写/目录/元数据操作，测试用 `recording`/内存实现，生产用 `LinuxFsIo`。

4.5 systemd 驱动层

`SystemdManager` 通过 `SystemdConfig { target, version, bus, default_destination }` 构造；`SystemdTarget` 为类型化 `Scope { name, slice } / Slice { name, parent }`。new 只连接并解析能力，start 显式启动 transient unit；另有 `open`、`signal`/`stop`/`reset_failed`、`unit`、`control_group`、`set_property` 等。

- 实际 cgroup 路径通过 D-Bus 查询 unit 的 `ControlGroup`，不按命名约定推导；统计可组合只读 cgroupfs observer；
- **不提供公共** `attach_thread()` (`systemd` 无法保持线程级语义，任何私有尝试必须返回 `Unsupported`)；**不提供文件系统式** `remove()`，`stop` 的进程与状态副作用由 `StopOptions`/`StopReport` 表达；
- 私有 `SystemdDialect` (`on-v1/v2`) 只负责资源→property 规划 (`plan_properties`)、能力解释与统计归一化；`SystemdBus` trait 负责 unit 生命周期与传输，测试用 `recording bus`；
- `SystemdProperty` 为自有已校验 owned 类型，raw `zbus` 类型保持私有。

4.6 OCI 适配层

```
pub fn from_oci(
    resources: &oci_spec::runtime::LinuxResources,
    capabilities: &Capabilities,
) → Result<ResourcePatch, OciTranslationError>;
```

适配层负责 OCI memory+swap → v2 swap、CPU shares → v2 weight 等值转换、sentinel 值校验，以及对所选版本/后端 unsupported 字段的显式报告。不得实现 Manager 生命周期、不得做文件系统或 D-Bus I/O、不得选择 cgroup 路径、不得检测运行时、不得静默抑制 unsupported 字段。后端支持与否由 `Capabilities` 预检并在 `apply` 时返回 `Unsupported`。

4.7 兼容层与迁移方案

兼容验证覆盖仓库内新旧两套 API：高层 `Manager` 接口 (`Box<dyn Manager>`、`set(&LinuxResources)`、进程/线程迁移、`cgroup_path()`、`systemd()/v2()` 分支) 与低层 `fs` 模块 (`Cgroup/Controller/fs::Resources` 类型)。迁移步骤：

1. 先引入核心类型和新接口，不立即删除旧 API，旧入口经兼容 facade/re-export/adaptor 调用新实现；
2. 旧字符串路径构造经 `CgroupPath` 校验转换 (checked conversion shim)，迁移期间不改变资源设置行为；
3. 新接口对 unsupported 字段返回错误；旧接口若需保留 `systemd` 静默忽略的历史行为，仅在兼容层实现并记录迁移期限；

4. 对确需移除的 API 提供 deprecated 标记、替代示例和移除版本；
5. 每个涉及公共 API 的阶段都保持旧接口编译兼容并补充典型调用测试；
6. DinD subgroup 替换为显式 Subgroup(CgroupPath::new("init")?) 或 SameCgroupAs(container_init_pid) 时，同步提供迁移示例与兼容入口；库不得仅凭存在 init 子目录推断运行时意图；
7. 模块边界按目标 workspace 设计（model/*、发现数据模型、manager.rs 契约 → cgroups-core；Linux 发现读取器、backend/cgroupfs/*、io/linux.rs → cgroups-fs；backend/systemd/* → cgroups-systemd；OCI 翻译 → cgroups-oci；兼容导出 → cgroups-rs），接口稳定后再做拆分；
8. 在 v1.0.0 前完成 API 审计，由维护者确认长期支持的公共接口。

五、错误处理设计

```
pub enum Error {
    InvalidPath { /* context */ },
    InvalidResource { /* context */ },
    NotFound { /* context */ },
    AlreadyExists { /* context */ },
    VersionMismatch { /* required and discovered */ },
    Unsupported { /* operation and capability */ },
    PartialApply(ApplyReport),
    Backend { /* backend, operation, target, source */ },
}
```

配套原则：

- 禁止在正常错误路径中使用 unwrap() / expect()；所有可能失败的文件、D-Bus 和解析操作返回 Result；
- 错误保留后端、cgroup 路径、控制器/文件（或 property）、操作类型及底层 OS 错误（source chain）；
- VersionMismatch 用于版本策略要求与发现结果不一致的场景，不强行改变宿主版本；
- ApplyReport 显式列出已应用、跳过、unsupported 与失败条目，按"先规划后执行"报告部分应用结果，不假装事务性回滚；
- 不支持的资源/操作统一返回 Unsupported，可由 Capabilities 预检，不得静默成功或忽略；
- 避免把"文件不存在"或读取/解析失败一律转换为空值、空列表、false 或零值；
- 为 cgroupfs 与 systemd 提供一致的上层错误语义，D-Bus 具体值类型保持后端私有。

六、测试方案

6.1 单元测试

- CgroupPath 校验（拒绝绝对路径、`..`、根路径、平台前缀）与 ProcessId / ThreadId 受检查转换；
- mountinfo/成员关系解析器 fixture 测试（v1/v2/hybrid、mount namespace、非标准挂载点、转义字段、含冒号路径、畸形记录不 panic、发现错误 vs v1 误判）；
- 资源配置到文件值或 systemd 属性的转换，v1 shares → v2 weight 等版本差异转换的集中测试；
- OCI Spec → ResourcePatch 转换（含 unsupported 字段报告）；
- cgroup 路径与 systemd unit 名称处理；错误类型、上下文与 ApplyReport 部分应用语义；Capabilities 快照；控制器边界值与非法输入。

通过临时目录、可注入 I/O 抽象（recording FsIo、recording bus）使大部分测试无需 root；后端契约测试针对 recording 实现验证写入序列与错误传播，不依赖真实内核。

6.2 集成测试

保留现有 QEMU 矩阵，覆盖 CgroupFsManager × {v1, v2} 与 SystemdManager × {on-v1, on-v2} 四套实现，并扩展：新旧接口生命周期对照；Subgroup / SameCgroupAs 在嵌套 v2（no-internal-process 约束）下的挂入行为（参考进程被移动后按挂入时解析）；systemd 启动失败补偿与 attach 时序；unsupported 行为一致性；OCI 适配层端到端；失败清理；新旧 API 的兼容构建与最小调用测试。特权测试（v1、v2、hybrid、systemd、delegated/rootless、cgroup namespace 与嵌套布局）单独成套，与普通测试分离。

6.3 覆盖率目标

核心模型、解析器、适配器和公共接口覆盖率 ≥ 80%（cargo-llvm-cov 统计，CI 生成报告）；依赖真实内核、D-Bus 或权限的代码单独统计，不计入阈值。

七、实施计划

第一阶段：基线确认、行为刻画与架构设计（第 1—2 周）

- 更新 upstream 主线、Issue 和活跃 PR 状态；搭建本地 v1/v2 与 systemd 环境；建立 feature/测试/覆盖率基线；
 - 绘制模块依赖与调用关系，梳理公共 API 到 Manager、FsManager 和 SystemdManager 的调用路径；审计 fs::Resources、统计类型与公共 API，识别可能构成 v1.0.0 破坏性变更的接口并记录必须保持兼容的部分；
 - 编写定义纯核心边界的架构决策记录（ADR）；为现有 v1/v2 解析与资源转换补充 fixture 固化行为；建立独立于普通测试的特权集成测试矩阵；
 - 设计核心资源模型复用/迁移方案与 ManagedCgroup 契约、capability、systemd 控制面/cgroupfs 观测边界；
 - 与导师确认项目页面 RISC-V64 标记是否属于验收范围。
- 交付物：RFC、API 草案、后端能力矩阵、兼容性验证清单、迁移任务拆分、解析/转换

fixture。

第二阶段：路径安全与发现统一（第 3—4 周，首个实现 PR）

- 新增 CgroupPath（含完整单元测试）；新增 HostPaths 与不可变 CgroupTopology；
 - 实现单一 mountinfo/成员关系解析器（转义字段、含冒号路径、畸形记录报错不 panic）并覆盖 fixture；
 - 现有 hierarchy 与 Manager 构造器改走统一发现；旧字符串构造入口经 CgroupPath 校验转换保留兼容；本阶段不改变资源配置行为。
- 交付物：CgroupPath、发现层、拓扑快照、兼容 shim 及单元测试。

第三阶段：移除运行时策略并引入 cgroup 原生 Manager API（第 5—7 周）

- 删除基于 init 子目录的自动 DinD 探测（subcgroup()）；
 - cgroupfs 新增类型化 AttachDestination，SameCgroupAs 在挂入时解析参考进程当前成员关系并校验位于被管理 cgroup 之下；
 - 移除核心模块与文档中的容器运行时术语；
 - 新增 ResourcePatch、Capabilities、StatsRequest 与结构化错误；实现 CgroupFsManager（new/open/create 分离）与私有 v1/v2 FsDialect；
 - 引入 ManagedCgroup 契约（含 inherent 方法）；membership 移回 cgroup 本身；stats() 改为可失败，资源更新返回 ApplyReport。
- 交付物：CgroupFsManager 与私有 dialect、ManagedCgroup 契约、cgroup 原生资源/能力/统计类型及测试。

第四阶段：systemd 后端迁移与 OCI 隔离（第 8—9 周）

- 新增类型化 SystemdTarget 与 SystemdProperty owned 类型，raw zbus 保持私有；
 - 通过 D-Bus 查询 unit 实际 ControlGroup；cgroup 原生资源更新映射到受支持的 systemd 属性（私有 SystemdDialect）；
 - systemd 不提供公共 attach_thread()，无法保持线程语义时返回 Unsupported；
 - 实现 OCI → ResourcePatch 翻译器，oci-spec 移出核心依赖图；弃用当前 OCI 耦合 Manager API；
 - 测试 OCI feature 开关构建；验证新旧 API 的兼容性；提供迁移示例。
- 交付物：解耦后的 systemd 控制面、OCI 适配器、兼容层、迁移示例和兼容性验证结果。

第五阶段：测试、CI 完善与可选 Issue 修复（第 10 周，与前述并行）

- 增加边界值与错误路径测试；在 QEMU 矩阵中补充四套实现的新接口与失败清理测试；
 - 完善 feature、文档与 API 兼容检查；
 - 若里程碑按期完成，选择维护者确认、无活跃 PR 覆盖且可闭环的 Issue 提交带回归测试的修复。
- 交付物：测试与 CI PR、覆盖率报告；可选为独立 Issue 修复 PR。

第六阶段：兼容清理、文档、API 审计与发布准备（第 11—12 周）

- 将旧 `fs::Cgroup`、`Builder` 和 `Controller` API 适配到新实现；删除重复发现/解析代码、未校验字符串路径和"错误默认为默认值"行为；
 - 补全英文 API 文档，更新 README 和示例，编写迁移指南；运行公共 API 与语义版本检查；
 - 若模块边界稳定，拆分为 workspace crates（`cgroups-core`、`cgroups-fs`、`cgroups-systemd`、`cgroups-oci`、`cgroups-rs`）；规划下一大版本破坏性移除项；
 - 整理 CHANGELOG；修复 API 审计与评审问题；协助维护者达到 v1.0.0 release-ready。
交付物：完整文档、迁移指南、API 差异报告、CHANGELOG 和 Release Checklist。
-

八、验收标准

架构重构

- 核心公共 API 不引用 OCI 或任何容器运行时；核心代码无 Docker/DinD/运行时自动检测（不依据 `init` 子目录推断行为）；
- `CgroupFsManager` 与 `SystemdManager` 是仅有的两个公共后端，共用 `ManagedCgroup` 契约；`cgroupfs v1/v2`、`systemd-on-v1/v2` 是独立受测的私有实现，而非公共 `Manager` 家族；
- 进程挂入支持 `Subgroup` 与 `SameCgroupAs(pid)`；所有被变更路径都由受校验的 `CgroupPath` 表达；
- 发现只有单一实现并支持 namespace 化挂载布局，拓扑经 `Arc<CgroupTopology>` 共享；
- 统计与 `membership` 读取保留错误；不支持的资源/操作返回明确错误，不能静默成功；`systemd` 线程挂入不得静默退化为整进程挂入；
- 现有 `fs::Resources` 等类型与新 `core` 模型的复用/兼容/废弃关系有明确文档，不新增未说明迁移关系的平行资源模型；
- `systemd` 控制面不再直接依赖具体 `FsManager`，通过明确接口组合必要的 `cgroupfs` 观测/操作能力；
- 常规单元测试与契约测试不需要 `root` 权限。

功能与 Issue 修复

- 核心重构中发现和修复的 Bug 均含可复现/回归测试；正常错误路径无新增 panic；
- 可选 Issue 修复不作为主线验收前置条件；若提交，每个修复必须含回归测试。

测试与 CI

- 核心模块覆盖率 $\geq 80\%$ ；QEMU 矩阵持续通过并覆盖四套实现的新接口关键路径；
- CI 增加文档、feature 和覆盖率检查；主要 feature 组合均可构建（含 OCI 开关）；
- `cargo test --doc /README/examples` 编译与 `cargo doc` 验证不依赖特权 runner；
- `cargo semver-checks` 或等价公共 API 差异报告；最低支持 Rust 版本构建；必要时增加旧 API 最小兼容构建；

- 高成本 QEMU 测试按 v1/v2 分层，通过缓存、变更路径判断或定期任务控制时长。

文档与发布

- OCI 翻译为可选能力且不执行任何 I/O；公共 API 有符合 cargo doc 标准的英文文档；README/examples 可编译；
 - 文档覆盖 cgroupfs、systemd、OCI 使用示例，v1/v2 差异与 feature flag 说明，以及错误处理示例；
 - 提供旧版 → v1.0.0 迁移指南；完成 CHANGELOG、版本检查和发布清单；达到维护者认可的 v1.0.0 release-ready 条件。
-

九、预期成果

项目完成后，cgroups-rs 将形成：

1. 基于现有公开资源类型演进、与 OCI Spec 解耦的核心资源模型（CgroupPath、ResourcePatch、Capabilities、可失败统计）；
2. CgroupFsManager 与 SystemdManager 两个公共后端、私有 v1/v2 dialect 与共用 ManagedCgroup 契约；
3. 单一 CgroupTopology 发现源与边界清晰的 cgroupfs 能力层、systemd 控制面；
4. 可选的 OCI Spec 适配层（只翻译、不 I/O）；
5. 更统一、可诊断的错误模型（Unsupported、VersionMismatch、PartialApply 等）；
6. 在现有 QEMU 矩阵上完善的 cgroup v1/v2 与 systemd 测试体系（四套受测实现，常规测试无需 root）；
7. 80% 以上的核心模块测试覆盖率；
8. 可编译的示例和完整英文 API 文档；
9. cgroups-rs v1.0.0 release-ready 所需的迁移指南、API 差异报告、CHANGELOG 和 Release Checklist。